ARCHITECTURE & DESIGN SPECIFICATION | ESP-IDF V5.X | FREERTOS DUAL-CORE | AUTOMOTIVE EMBEDDED

Driver Drowsiness Detection & Emergency Response System

A Deterministic Dual-Core Firmware Architecture for Physiological Respiratory Monitoring, Vehicle Deceleration, and IoT Cloud Escalation

TARGET HARDWARE	FRAMEWORK	CONCURRENCY MODEL	DOCUMENT STATUS
ESP32-WROOM-32	ESP-IDF v5.x (Pure C)	FreeRTOS Dual-Core	Approved Architecture

1. Executive Summary & System Architecture

Drowsiness while operating motor vehicles accounts for significant road fatalities worldwide. This project implements a safety-critical embedded firmware solution engineered to run on the Espressif ESP32 dual-core microcontroller. By fusing physiological respiratory rate (measured via chest strap flex/FSR sensor) with driver steering dynamics (via analog angular displacement), the system detects micro-sleep episodes, hypopnea, and driver unconsciousness with high precision.

The system follows an automated escalation protocol: immediate audible/visual feedback in **WARNING** state, controlled PWM deceleration down to complete vehicle stoppage in **CRITICAL** state, followed by autonomous emergency coordinate transmission via GSM SMS and MQTT cloud broker telemetry in **EMERGENCY** state.

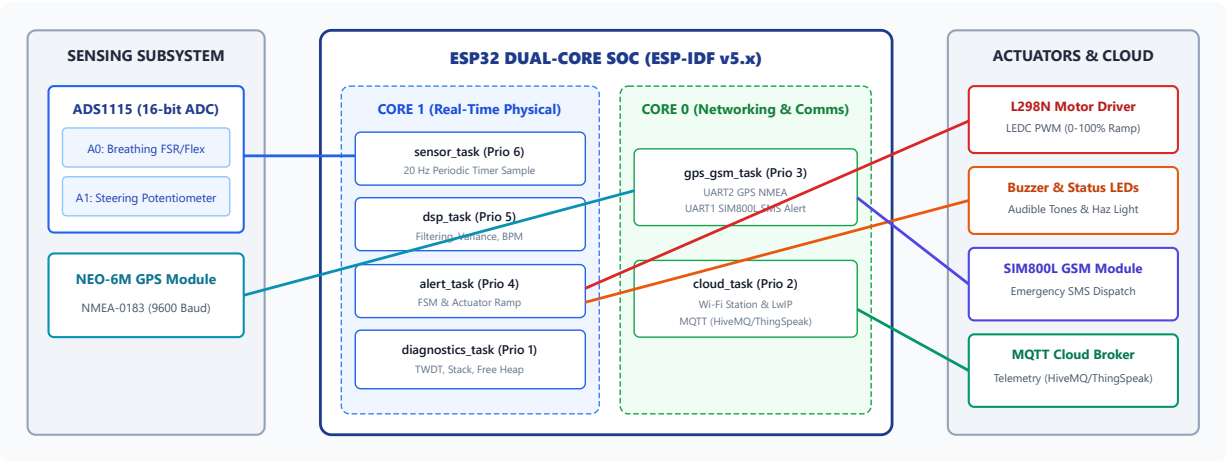


Figure 1.1: Complete Hardware Interconnect and Dual-Core FreeRTOS Task Architecture

Hardware Pinout & Peripheral Allocation Matrix

Peripheral	ESP32 Pin	Signal / Mode	Hardware Wiring Notes & Electrical Protection
I2C Master	GPIO 21 (SDA) GPIO 22 (SCL)	I2C Bus (100 kHz Standard Mode)	ADS1115 ADC address `0x48` (ADDR to GND). External 4.7 kΩ pull-ups to 3.3V. Ch A0 = Respiratory divider, Ch A1 = Steering wiper.
L298N Motor	GPIO 18 (PWM) GPIO 19 (IN1) GPIO 5 (IN2)	LEDC Channel 0 Standard GPIO Out	LEDC PWM configured at 20 kHz, 10-bit resolution (0-1023). IN1/IN2 set direction forward/brake. L298N logic powered from 5V rail.

Peripheral	ESP32 Pin	Signal / Mode	Hardware Wiring Notes & Electrical Protection
Buzzer & LED	GPIO 23 (Buzzer) GPIO 2 (LED_WARN) GPIO 4 (LED_CRIT)	GPIO Out / PWM Tones	Active/Passive buzzer driven via 2N2222 NPN transistor with flyback diode. LEDs current-limited via 330 Ω series resistors.
NEO-6M GPS	GPIO 16 (RX2) GPIO 17 (TX2)	UART Port 2 (9600 Baud, 8-N-1)	NEO-6M TX connected directly to ESP32 RX2 (3.3V tolerant). NMEA sentences parsed via ring buffer.
SIM800L GSM	GPIO 26 (TX1) GPIO 27 (RX1)	UART Port 1 (115200 Baud, 8-N-1)	**Crucial:** SIM800L requires 3.7V - 4.2V (2A peak burst current). Requires external step-down regulator (LM2596/Buck) with 1000 µF capacitor. Common GND with ESP32.

2. FreeRTOS Tasks, Priority Justification & Core Pinning

The FreeRTOS priority scheme is engineered to guarantee **zero sample starvation** and eliminate priority inversion. Core 1 is reserved exclusively for real-time sensing and motor actuator control, insulating high-frequency digital signal processing from the non-deterministic network stack (LwIP / Wi-Fi / GSM AT delays) operating on Core 0.

Task Identifier	Core	Priority	Stack	Period / Trigger	Engineering Rationale & Latency Budget
`sensor_task`	Core 1	6 (High)	3072 B	Periodic 50 ms (20 Hz)	Uses `vTaskDelayUntil()` for absolute period consistency. High priority ensures < 0.5 ms sample jitter. Pushes raw samples into `sensor_queue`.
`dsp_task`	Core 1	5	4096 B	Queue-driven (Per Sample)	Blocks on `sensor_queue`. Performs windowed moving average, variance calculation, and refractory peak detection. Execution time < 2.1 ms per sample.
`alert_task`	Core 1	4	3584 B	Queue-driven / 100 ms Tick	Evaluates FSM transitions, executes soft PWM ramping on motor, and drives buzzer cadences. Sets emergency bits in `system_event_group`.
`gps_gsm_task`	Core 0	3	4608 B	Event / Emergency Trigger	Continuously buffers valid GPS fixes from UART2. Upon receiving `EVENT_EMERGENCY_TRIGGER`, formats and dispatches SMS via SIM800L AT engine.
`cloud_task`	Core 0	2	5120 B	Periodic 2s / State Change	Manages MQTT TLS/TCP connection, encodes JSON telemetry payloads (State, BPM, Lat, Lon, Speed), and handles broker keep-alives without blocking Core 1.
`diagnostics_task`	Core 1	1 (Low)	3072 B	Periodic 5000 ms	Monitors task stack high-water marks (`uxTaskGetStackHighWaterMark`), free heap, resets TWDT watchdog, and computes sensor-to-actuator latency.

Inter-Task Data Flow & Queue Architecture

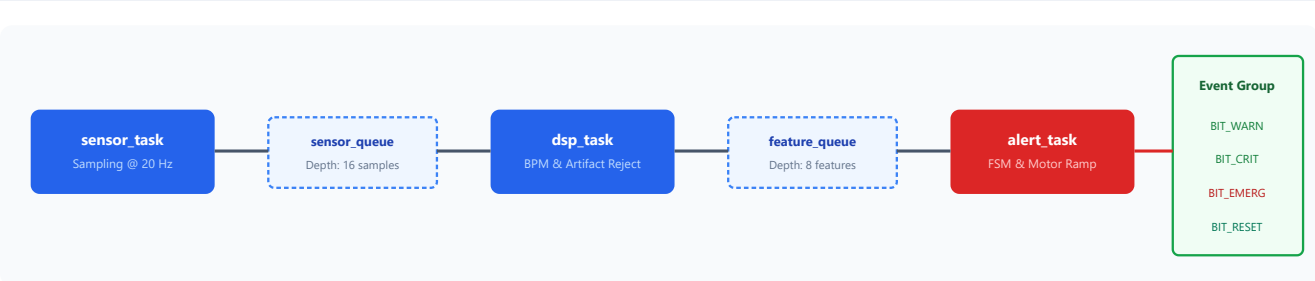


Figure 2.1: Lock-Free FreeRTOS Queue Streaming Pipeline

3. Finite State Machine (FSM) & Emergency Escalation Protocol

The system state machine follows strict deterministic safety state transitions. A critical safety requirement is that **any state can immediately transition back to NORMAL** if the driver exhibits confirmed active recovery (steering wheel deflection combined with restored rhythmic breathing).

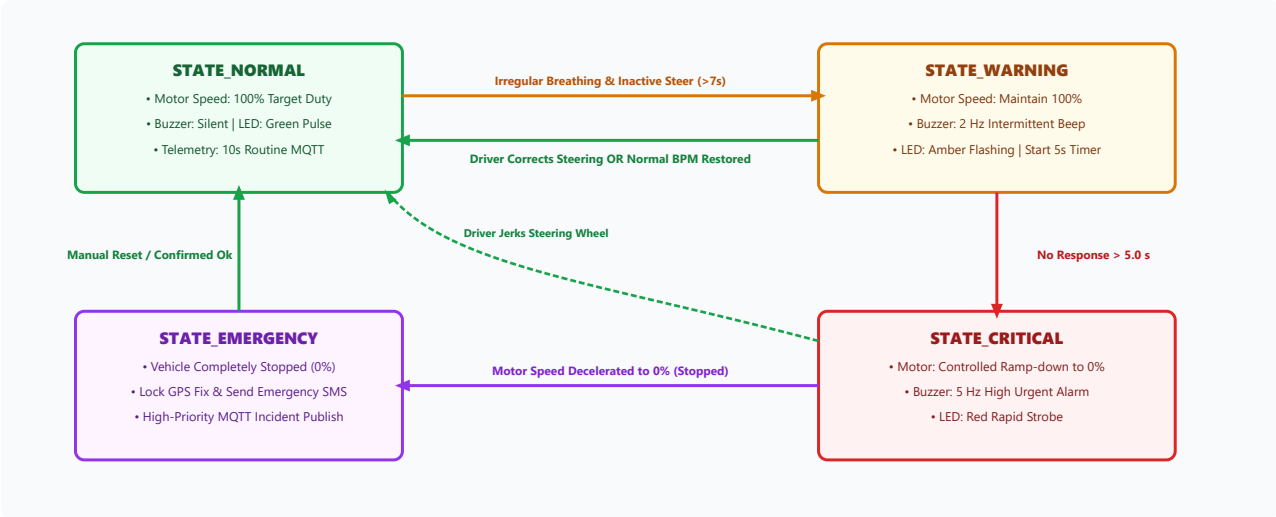


Figure 3.1: Hierarchical State Transition Flowchart with Rapid Driver Recovery Paths

State Escalation & Actuator Response Specifications

State	Entry Condition	Motor Actuator Profile	Audio/Visual Pattern	IoT Escalation Action
NORMAL	Normal breathing (10-30 BPM) & active steering detected.	Full Target PWM (100% duty cycle, 1023 count).	Buzzer silent. Green LED heartbeat pulse (1 Hz).	Periodic MQTT telemetry (interval: 10 seconds).
WARNING	BPM outside [10, 30] OR no peak > 8s **AND** steering inactive > 7s.	Maintain cruising speed. No motor braking yet.	Buzzer 2 Hz intermittent pulses. Amber LED flashing (2 Hz).	Immediate MQTT state update: `WARNING` alert.
CRITICAL	Warning state active for > 5 seconds with zero driver corrective input.	Linear PWM ramp-down (LEDC fade: 100% → 0% in 4.0 sec).	Continuous 5 Hz piercing buzzer. Red strobe LED (5 Hz).	Immediate MQTT telemetry: `CRITICAL_DECEL`.
EMERGENCY	PWM reaches 0% (vehicle confirmed stationary).	LEDC duty 0%. Reverse braking polarity applied for 200 ms.	High-pitch alternating siren. Dual LED hazard strobe.	Read latest GPS NMEA fix. Transmit emergency SMS via SIM800L. Publish MQTT.

4. Digital Signal Processing (DSP) Pipeline Design

Biomedical signal acquisition in automotive environments experiences severe mechanical noise (road bumps, suspension vibration, seat shifts). The DSP pipeline combines low-pass signal conditioning, sliding window variance for motion artifact rejection, and refractory peak detection.

1. Temporal Sampling & Low-Pass Filtering

Human breathing ranges from 0.15 Hz to 0.50 Hz (9 to 30 BPM). Sampling at $f_s = 20\ Hz$ provides 40× oversampling. A moving average filter of window $W = 10$ samples (500 ms) removes high-frequency electrical and mechanical noise:

$$y[n] = (1 / W) \times \sum_{k=0..W-1} x[n-k]$$

Combined with single-pole IIR baseline smoothing: s[n] = α × y[n] + (1 - α) × s[n-1] (α = 0.15).

2. Motion Artifact Rejection (Variance Gating)

Vehicle jolts and road bumps create rapid high-amplitude voltage spikes that can corrupt peak detection. We compute rolling variance over a 1-second sliding buffer ($N = 20$ samples):

$$\sigma^2 = (1 / N) \times \sum_{i=0..N-1} (x_i - \bar{x})^2$$

If $\sigma^2 > TH_{artifact}$, the window is flagged as corrupted; peak detection is gated to eliminate false apnea triggers.

3. Adaptive Peak Detection & BPM Calculation

A peak is registered when y[n-1] > y[n] and y[n-1] > y[n-2] and y[n-1] > $Threshold_{adaptive}$.

- **Refractory Period:** $T_{refractory}$ = 1.2 s (≤ 50 BPM) prevents multiple triggers on a single breath.
- **Interval Measurement:** $\Delta t = t_{peak}[k] - t_{peak}[k-1]$.
- **Instantaneous BPM:** BPM = 60 / Δt. Outliers filtered via 4-sample median window.

4. Steering Inactivity & Apnea Criteria

Steering sensor angle is sampled at 20 Hz. We evaluate cumulative displacement |Δθ| over a 7-second sliding window.

- **Steering Inactivity:** Flagged if |Δθ| < $DEADBAND_{threshold}$ continuously for > 7 s.
- **Apnea / Cessation:** Flagged if no valid respiratory peak occurs for > 8.0 s.
- **Tachypnea / Hyperventilation:** Flagged if BPM > 30.

5. ESP-IDF Modular Component Architecture

The code is strictly decoupled into reusable ESP-IDF components conforming to the ESP-IDF v5.x build system.

```
driver_drowsiness_system/
├── CMakeLists.txt                    # Project-level CMake (ESP-IDF v5)
├── Kconfig.projbuild                 # Simulation mode toggle & runtime tuning
├── sdkconfig.defaults                # Pre-configured FreeRTOS, TWDT, and Stack sizes
├── components/
│   ├── system_config/                # Global types, pin definitions, queue handles
│   │   ├── include/system_config.h   # GPIO mapping, baud rates, threshold constants
│   │   └── include/system_types.h    # sensor_raw_data_t, dsp_features_t, system_state_t
│   ├── ads1115/                      # I2C driver for ADS1115 16-bit ADC (A0/A1 continuous read)
│   ├── buzzer_led/                   # Audible alert and status LED driver
│   ├── motor_pwm/                    # L298N LEDC hardware PWM speed ramp engine
│   ├── gps_nmea/                     # UART2 driver + zero-allocation NMEA parser ($GPRMC)
│   ├── sim800l/                      # UART1 driver + AT command GSM SMS emergency dispatcher
│   ├── dsp_pipeline/                 # Low-pass filter, variance calculation, BPM peak detector
│   ├── state_machine/                # FSM implementation, timeout timers, and recovery logic
│   └── cloud_telemetry/              # Wi-Fi Station + MQTT client (HiveMQ / ThingSpeak JSON)
├── main/
│   ├── CMakeLists.txt
│   └── main.c                        # System init, task launch, TWDT registration, diagnostics
└── test/
    └── host_dsp_test/                # Host-native C unit tests (GCC/Clang)
```

```
├── Makefile                    # Standalone host test runner
├── test_dsp.c                  # Unit tests for filter, BPM, and artifact rejection
└── test_data.h                 # Synthetic physiological datasets (normal, apnea, noise)
```

6. Simulation Engine & Host Unit Testing Strategy

To facilitate comprehensive testing without requiring physical sensors, the project includes an internal **Kconfig-controlled Simulation Engine** as well as standalone **Host-side C Unit Tests** runnable on any Linux/macOS/Windows GCC environment.

Kconfig Hardware Simulation Mode

When `CONFIG_ENABLE_SIMULATION_MODE=y` is enabled, `sensor_task` bypasses physical I2C ADS1115 reads and injects mathematically synthesized physiological data:

- **Phase 1 (0 - 30s):** Normal rhythmic breathing (16 BPM) with continuous micro-steering corrections.
- **Phase 2 (30 - 45s):** Road bump vibration noise injected (σ^2 > TH, verifying artifact rejection).
- **Phase 3 (45 - 60s):** Apnea onset (0 BPM) + completely static steering wheel. Escalates system into `WARNING` → `CRITICAL` → `EMERGENCY`.
- **Phase 4 (60s+):** Simulated driver steering correction, demonstrating immediate recovery to `NORMAL`.

Host-Native DSP Unit Test Suite

Located under `test/host_dsp_test/`, compiled using native GCC without ESP-IDF dependencies:

- `test_filter_attenuation()`: Verifies > 20 dB attenuation of 5 Hz engine vibration noise.
- `test_variance_artifact_rejection()`: Verifies peak detector pauses during large shock inputs.
- `test_bpm_accuracy()`: Validates calculated BPM matches known synthetic ground truth within ± 1 BPM.
- `test_apnea_detection()`: Asserts apnea flag triggers exactly 8.0 s after breath cessation.

7. Instrumentation, Task Watchdog & Memory Safety

Task Watchdog Timer (TWDT)

Configured via `esp_task_wdt_init()` with a 3.0-second timeout. All FreeRTOS tasks (`sensor_task`, `dsp_task`, `alert_task`, `gps_gsm_task`, `cloud_task`) register with TWDT and feed the watchdog on each execution loop. If any task blocks indefinitely (e.g. frozen UART or hung I2C), TWDT triggers an ESP32 reboot.

Microsecond Latency Instrumentation

Using the ESP32 hardware cycle counter (`esp_cpu_get_ccount()`), we benchmark the exact duration from ADC sample acquisition to actuator response:

$$\text{Latency} = (\text{ccount}_{\text{alert}} - \text{ccount}_{\text{sensor}}) / 240{,}000{,}000 \text{ seconds}$$

Reported periodically by `diagnostics_task` alongside stack high-water marks.

8. Phased Implementation Roadmap

Phase	Deliverables & Code Additions	Verification & Test Criteria	Interview Defense Focus
Phase 1: Foundation & Drivers	Scaffolding, Kconfig, `system_config`, `ads1115`, `buzzer_led`, `motor_pwm`, Synthetic Simulation generator.	PWM duty cycle ramp test, I2C ADC reading verification, synthetic signal generation.	I2C register configuration, LEDC PWM resolution vs frequency tradeoff, GPIO current limits.

Phase	Deliverables & Code Additions	Verification & Test Criteria	Interview Defense Focus
Phase 2: DSP & Host Tests	`dsp_pipeline` component (moving average, variance, peak detector, BPM, steering deadband), host test suite.	Run native host unit tests: GCC compilation, assert zero test failures on BPM & artifacts.	Filter latency, Nyquist criteria, variance sliding window math, fixed-point vs floating-point on Xtensa.
Phase 3: FSM & Alert Engine	`state_machine`, `sensor_task`, `dsp_task`, `alert_task` on Core 1 with FreeRTOS queues.	End-to-end escalation from simulated apnea to PWM deceleration and buzzer alarm. Recovery test.	FreeRTOS priority justification, core affinity, queue depth sizing, race condition prevention.
Phase 4: Comms & Cloud	`gps_nmea` parser, `sim800l` AT engine, `cloud_telemetry` (Wi-Fi + MQTT client) on Core 0.	NMEA sentence parsing validation, simulated AT command SMS dispatch, MQTT payload publishing.	Non-blocking UART buffering, GSM burst current mitigation, MQTT QoS levels, Wi-Fi reconnection logic.
Phase 5: Diagnostics & Packaging	`diagnostics_task`, TWDT integration, latency benchmarking, complete `README.md`.	Stack high-water mark printout, heap integrity, end-to-end latency measurement.	Watchdog architectures, memory leak prevention, embedded profiling methods.